

Sample Midterm 1 and *Solution Sketches*

In all problems, we fix the alphabet to be $\Sigma = \{0, 1\}$. Total points = 50.

Problem 1 (True or False)**15 points**

Answer each question with *only* a True (**T**) or False (**F**). Expect 10 questions like this:

1. _____ Every regular language is decidable by a Turing machine.

Answer: True. (DFAs can be viewed as a special case of Turing machines.)

2. _____ If A is a regular language and B is a language recognizable by an NFA, then $A \setminus B = \{x : x \in A \text{ and } x \notin B\}$ is the language of some regular expression.

Answer: True. (Recall that Regular = DFA = NFA = RE; and so $A \setminus B = A \cap \overline{B}$ is regular by the closure property of regular languages; Applying Regular = RE, we know that $A \setminus B$ has a regular expression.)

Problem 2 (Construct a DFA and RE)

10 points

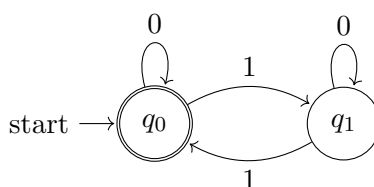
Construct a deterministic finite automaton (DFA) *and* a regular expression (RE) for each of the following languages:

- Expect 2 languages like this:

$$L_1 = \{w \in \{0,1\}^* : \text{there is an even number of 1s in } w\}$$

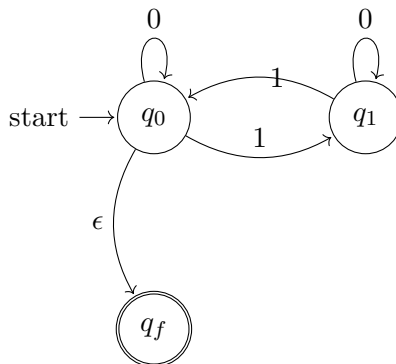
Answer:

DFA:

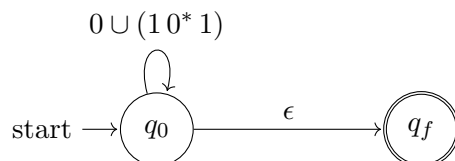


RE: $(0 \cup (10^*1))^*$, or any correct answer e.g. $0^*(10^*10^*)^*$. A RE can be obtained by converting DFA to GNFA and then to RE:

Step 1: add an accepting node q_f .



Step 2: Removing q_1 .



Step 3: Removing q_0 to get a RE: $(0 \cup (10^*1))^*$.

Problem 3 (Construct a NFA)

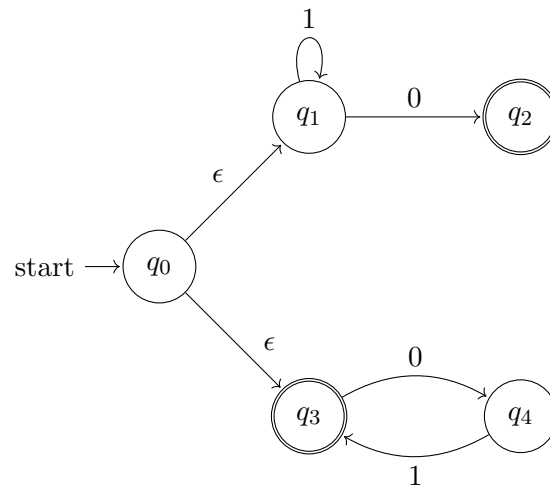
10 points

Construct a NFA for the following languages:

- **Expect 2 languages like this:**

The language given by the regular expression $1^*0 \cup (01)^*$.

Answer:



Problem 4 (Pumping Lemma)

10 points

Recall that to prove that a language L is not regular, the pumping lemma requires you to give a strategy for winning the following game :

- The adversary chooses some integer $p \geq 0$.
- You choose a string $w \in L$ such that $|w| \geq p$.
- The adversary chooses strings x, y, z such that $w = xyz$ and $|xy| \leq p, |y| \geq 1$.
- You choose an integer i and you win if xy^iz is not in L .

Use the pumping lemma to prove that the following languages are not regular:

1. **Expect 2 languages like this:** $L_1 = \{0^n 1^m : 0 \leq m \leq 10n\}$

Answer: For any $p \geq 0$, choose the string

$$w = 0^p 1^{10p}.$$

clearly $w \in L_1$ and $|w| \geq p$.

For any $w = xyz$ with $|y| \geq 1$ and $|xy| \leq p$, the substring xy consists entirely of 0's. Let $y = 0^k$ for some k with $1 \leq k \leq p$.

Now, choose $i = 0$. Then the pumped string is:

$$xy^0z = xz = 0^{p-|y|} 1^{10p}.$$

Since $|y| \geq 1$, this string can't be in L_1 because $10p \leq 10(p - |y|)$ doesn't hold.

Hence, L_1 is not regular.

Problem 5 (Turing machines)

5 points

Expect one problem like this:

Give a “tape-head level description” of a Turing Machine that recognizes the language $L = \{0^n 1^{n^2} : n \geq 1\}$. Do not draw a state diagram, but carefully describe the strategy of how to move on the tape, what to write on the tape (including special symbols you may use and what they denote), etc., like we did in class. (You can use multi-tape Turing machines, but not random access Turing machines.)

Answer: Observe that for any string $w = 0^n 1^{n^2}$ in the language, we can view the string as having n zeros followed n “chunks” of n ones each. Our strategy is to first check that the number of ones in the string can be split into chunks of size n and then check that there are n total chunks. We define our tape-head level description as follows.

1. Check that the string is of the form $0^+ 1^+$, if not, halt and reject.
2. Return to the beginning of the string. Cross out the first 0, then walk right until we encounter a (fresh) 1. Mark that 1 with a star and then walk back left until we hit a crossed out 0.
3. Walk back and forth along the string, crossing out one 0 and one corresponding 1 at a time (do not mark these 1's with a star, just cross them out). If we ever cross out a 0 and then reach the right end of the string without finding an uncrossed 1, halt and reject.
4. Once we've run out of 0's to cross out, return to the beginning of the string and uncross all the 0's.
5. Repeat steps 2-4 until we've crossed out all of the 1's. We've now divided the 1's into chunks where each chunk begins with a starred 1, and has as many 1's as there are 0's.
6. Return to the beginning of the string, then walk back and forth along the string crossing out one 0 and one corresponding starred 1 at a time. If we run out of 0's before we run out of starred 1's or vice versa, reject. Otherwise accept.

Alternative answer: Another potential strategy is to do a “double loop” over the 0's by keeping two special pointers: an inner pointer X and an outer pointer Y that we can put on the tape. Initially we place both pointers on the first 0 on the tape (we can use a special symbol ‘ XY ’ on the tape to represent both pointers being in the same location). We repeat a step where we cross out one 1 and move the inner pointer X to the next 0 on the right. When the inner pointer reaches the end of the 0's we move it all the way left and move the outer pointer Y one to the right. We accept if both pointers are all the way at the right of the 0's and we crossed out all the 1's. If we run out of 1's before than or both pointers are on the right and there are still 1s left, we reject.